OpenClaw 个人 Mini 实现——Python 智能体编程作业

一、作业背景与目标

OpenClaw 是 2026 年爆火的 MIT 开源 AI 智能体框架。它本身不生产大模型，而是作为连接各大模型（OpenAI、Claude、DeepSeek 等）与真实系统操作之间的“中枢”——让 AI 从“只会说话”变成“会动手做事”，能够完成文件处理、信息检索、自动化流程等多种任务。

OpenClaw 的核心其实是一个经典的“消息处理循环”：接收输入 → LLM 分析并决策（回复 or 调用工具）→ 执行工具并获取结果 → 将结果反馈给 LLM → 生成最终回复。港大黄超老师课题组甚至将其重构为仅约 4000 行的 nanobot，保留了完整功能的同时大幅降低了理解门槛。

本次作业的目标是：**在理解 OpenClaw 核心原理的基础上，自己动手用 Python 编写一个“迷你版 OpenClaw”——一个能够接收用户指令、调用大模型进行推理、根据推理结果调用本地工具并执行、最终完成任务的极简智能体框架。**

二、作业要求

2.1 整体框架要求（必做）

你需要实现一个 Python 程序，具备以下核心能力：

（1）大模型接入与对话能力

- 支持接入至少一种大模型 API（如 OpenAI GPT / Claude / DeepSeek / 阿里云百炼等）
- 能够接收用户输入的自然语言指令
- 能够与大模型进行多轮对话（维护对话历史上下文）

（2）工具调用（Function Calling）能力

- 至少实现 **3 个自定义工具函数**，每个工具能够完成一项具体的本地操作
- 大模型能够根据用户意图，自主判断是否需要调用工具、调用哪个工具、传入什么参数
- 工具执行结果能够被正确获取并返回给大模型

（3）ReAct 推理循环

- 实现一个基础的消息处理循环（推荐采用 ReAct 模式：Thought → Action → Observation）
- 支持多轮工具调用（一个复杂任务可能需要多次调用工具）

- 设置合理的终止条件（如 LLM 给出 final_answer 或达到最大轮数）

（4）本地执行与安全隔离

- 工具执行时应有基本的安全检查（如操作范围限定在指定目录内）
- 需要对工具调用过程中的异常进行捕获和友好提示

2.2 扩展建议

- 对话历史持久化：将对话记录保存到本地 JSON 文件，下次启动时可加载历史
- 工具插件化：支持通过配置文件或装饰器动态注册新工具，无需修改核心代码
- 流式输出：支持 LLM 的流式响应，逐字输出（而不是一次性返回）
- 多轮规划展示：在控制台清晰展示每轮的“思考→行动→观察”过程
- 安全的 shell 执行沙箱：对 run_shell_command 实现更严格的命令白名单和参数校验
- 支持多 Agent 协作雏形：实现一个简单的“主 Agent + 子 Agent”模式（如主 Agent 可将文件分析任务委托给专门的分析子 Agent）

三、作业要求

作业提交：4 月 30 日检查作业